

Domain 2: Tool Design & MCP Integration — Practice-Based Self-Questioning

1. Tool returns a 200-field customer record

- Self-question: Should the tool return only the commonly-needed fields concisely, with an optional parameter or follow-up call for more detail — rather than dumping the full record or forcing one call per field?
- Key distinction: concise-with-optional-detail vs. full record ("never lacks info") vs. one call per field vs. full record + "ignore irrelevant fields" prompt — which balances context economy against completeness?

2. Agent tricked into a \$1,500 refund by a claimed prior approval

- Self-question: Is the reliable fix enforcing the \$200 limit *server-side in the tool implementation*, rather than a system-prompt rule, fine-tuning, or a double-confirmation?
- Key distinction: server-side deterministic enforcement vs. system-prompt rule (probabilistic, defeatable by social engineering) vs. fine-tuning vs. asking the customer to confirm — which cannot be talked around?

3. Agent fabricates an order_id instead of calling lookup_order first

- Self-question: Does updating the tool *description* to state that order_id must come from a prior lookup_order call directly address the root cause of fabrication?
- Key distinction: description fix (root cause: the model's tool-selection signal) vs. forcing a tool call with **any** vs. server-side existence validation (catches it but doesn't prevent it) vs. pre-extracting IDs — which addresses *why* the model invented the value?

4. Communicating MCP tool errors back to the agent

- Self-question: Is the correct pattern returning the error in the tool result content with **isError: true** — rather than a success response with a status field, an empty result, or throwing an exception?
- Key distinction: **isError: true** in tool result vs. success-with-status-field vs. empty result (silent failure) vs. throwing an exception — which lets Claude recognize and reason about the failure?

5. Transient vs. permanent errors, agent wastes retries on permanent ones

- Self-question: Should errors be returned as structured responses with **retriable: false** for business errors plus a customer-friendly explanation — rather than auto-retrying, few-shot text parsing, or an eligibility pre-check tool?
- Key distinction: structured retriability + explanation vs. tool-level auto-retry vs. few-shot error-text parsing vs. adding a pre-check tool — which stops wasted retries *and* improves the customer-facing message?

6. process_refund times out but eligibility is verified

- Self-question: Should the agent explain the billing, confirm eligibility, acknowledge the system issue, and offer escalation/retry — rather than escalating immediately, confirming a refund it can't complete, or looping on retries?
- Key distinction: honest partial resolution + offer vs. immediate escalation vs. falsely confirming completion vs. blind retry-with-backoff keeping the conversation open — which balances first-contact resolution with honest error handling?

7. Tool count grew 4→10, semantic overlap dropped selection accuracy

- Self-question: Does *consolidating* the overlapping tools (merge `issue_credit/process_refund` with an action param; fold `check_delivery_status` into `lookup_order`) structurally eliminate the overlap — rather than deferring loading, few-shot, or splitting into sub-agents?
- Key distinction: structural consolidation (removes the ambiguity) vs. tool-search deferred loading vs. few-shot disambiguation vs. sub-agent split — which eliminates the *root cause* (two tools competing) rather than masking it?

8. Uniform "Operation failed" errors cause inconsistent retry/escalate

- Self-question: Is the fix enriching error responses with structured metadata (`errorCategory`, `isRetryable`, `description`) — so the agent can branch — rather than an `analyze_error` tool, few-shot parsing, or blanket retries?
- Key distinction: structured error metadata vs. a separate `analyze_error` tool vs. few-shot error-pattern interpretation vs. server-side retry-for-all — which gives the agent a reliable basis to distinguish error types?

9. "Tool execution failed" — following the documented recommendation

- Self-question: Does returning error-type-specific messages (with `is_error: true`) that say what went wrong and what to try next follow Anthropic's "instructive error messages" guidance?
- Key distinction: instructive per-type messages vs. an interception layer that tags errors before Claude sees them vs. removing `is_error` and treating errors as data vs. hiding transient errors via in-tool retry — which follows the documented recommendation *and* informs recovery?

10. Refunds over \$500 must always escalate (3% violation rate)

- Self-question: Is guaranteed compliance achieved with a hook that intercepts and blocks the tool call when the amount exceeds \$500 — rather than a tool error message, few-shot examples, or stronger prompt language?
- Key distinction: deterministic hook (guaranteed) vs. tool returning an error (relies on model to then escalate) vs. few-shot vs. emphatic prompt language — which provides an actual *guarantee* rather than reduced probability?

11. Ensuring metadata extraction runs before enrichment

- Self-question: Should `tool_choice` be set to force `extract_metadata` and enrichment handled on subsequent turns — rather than `any`, reordering tools under `auto`, or forcing `extract_metadata` on every call?
- Key distinction: force the specific tool then continue vs. `any` + prompt priority vs. `auto` + reordering (Claude doesn't prioritize by order) vs. forcing it on every call (breaks enrichment) — which guarantees ordering without blocking later steps?

12. Agent prefers Grep over an MCP analyze_dependencies tool

- Self-question: Does expanding the MCP tool's *description* to detail its capabilities and outputs improve selection — since Claude weighs description over name, and Grep's description is far more detailed?
- Key distinction: richer tool description vs. removing Grep vs. splitting into granular tools vs. system-prompt routing rules — which fixes the actual selection signal (a thin description losing to a detailed one)?